CS496 Software Project Proposal: Glory Harbor Works

Crystal Ajayi & Zoe Willis

2026-04-15

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

1. Client name: Victor Akinde
2. Client title: Pastor of STGCI-Glory Harbor
3. Client email address: stgciglory@gmail.com
4. Client employer: STGCI-Glory Harbor
5. How you know the client: Pastor Victor oversees the church that Crystal attends.

2 Project Description

2.1 Overview

For the project, we will create a website that centralizes many important church related functions into one accessible platform. In Crystal's experience serving in church in the media department, she has seen a need for better organization, clearer communication, and streamlined scheduling. There is also a need for a more modern way to share sermons, support members through prayer requests, and engage children with age-appropriate learning materials. This website aims to solve these challenges by giving the church a structured and easy-to-use tool that strengthens community involvement and improves internal operations.

2.2 Key Features

These are the key features of the software system the client would like us to build:

- **Sermon Media Hub:** Upload and stream audio/video sermons, with the ability to search through the videos by speaker, topic, or series.
- **Prayer Request System:** Allow users to submit public or anonymous requests, interact through an "I prayed for this" feature, and enable pastors to manage and categorize incoming requests.
- **Children's Ministry Learning Center:** Provide weekly lessons, quizzes, memory verses, and progress tracking designed for younger members.
- **Pastoral Appointment Scheduling:** Allow members to book one-on-one meetings with pastors, with automated confirmations and reminders.
- **User Accounts and Roles:** Secure login system with different permissions for members, leaders, and pastoral staff.

- **Push Notifications:** Alerts for new sermons, prayer updates, scheduled meetings, or new children's lessons.

2.3 Why this Project is Interesting

When deciding what to tackle for the capstone project, we knew we wanted to create something meaningful that would have a real an impact on others. This project is important because it allows us to directly support a congregation and provide a tool that improves communication, organization, and learning within the church. The challenges addressed by the website can also be used in other congregations that face similar needs in the future. The combination of practical needs, technical components, and personal motivation makes it a well rounded and exciting capstone project.

2.4 Areas of CS required

The areas of computer science the project will use includes:

- Mobile Application Development – for building a cross-platform church mobile app.
- Database Systems – for storing sermons, prayer requests, user accounts, lesson progress, and scheduling data.
- Backend/Web Services – to create APIs for authentication, sermon delivery, prayer management, and scheduling.
- Cloud Computing – for hosting media content, databases, authentication services, and storage.
- Information Security – to protect user data, prayer confidentiality, children's learning records, and secure user authentication.
- Human–Computer Interaction (HCI) – to design an accessible, intuitive interface for a diverse user base including kids, adults, and elderly members.
- Software Engineering – for requirements analysis, design, testing, documentation, and project management.

2.5 Potential Concerns and Questions

The scope of this project is a bit daunting. We will need to spend more hours working on it compared to the project for Software Engineering as there are fewer people working on it at once. We'll have to commit a considerable amount of time to it outside of the time we must spend on other assignments. In addition to that, Zoe does not come from a religious background and is uncertain if that will affect her understanding of the project.

2.6 Comparison to Draft

This project is the same one that Crystal proposed with some adjustments. It has the same client and the same basic overview. The main change of the project is the switch from an app to a website. The learning curve of trying to create an app with zero prior experience is too high for us to adapt to in the amount of time we have. Also based on the feedback from the proposal, we removed the stream audio/video sermon function and the machine learning aspect. There's a possibility they'll be too difficult to accomplish so we scrapped those ideas all together.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
NFR1	Access Control	Security	The system shall enforce role-based access control to ensure users can only access features permitted by their assigned roles.
NFR2	Session Security	Security	The system shall automatically log users out after a period of inactivity to prevent unauthorized access to accounts.
NFR3	Data Persistence	Reliability	The system shall reliably save and recover user data to prevent data loss in the event of system restarts or failures.
NFR4	Consistent Layout	Usability	The system shall maintain a consistent layout and design across all pages to improve ease of use and navigation.
NFR5	Color Scheme Consistency	Usability	The system shall follow a consistent color scheme to ensure the website is visually appealing and easy to read.
NFR6	Mobile Compatibility	Portability	The system shall be mobile-friendly and compatible with different devices and screen sizes.
NFR7	Accessibility Compliance	Accessibility	The system shall be accessible to users with disabilities by supporting assistive technologies and accessibility best practices.

Table 1: Non-Functional Requirements

3.2 Functional Requirements (User Stories)

ID	Story Title	Points	Description
M1	Create Account	2	As a Member, I want to create a profile, so that I can access the website as a member.
M2	View Account	2	As a Member, I want to view my profile, so that I can look at my information
M3	Update Account	2	As a Member, I want to update my profile, so that I can change any info if needed.
M4	Delete Account	2	As a Member, I want to delete my profile, so that I can no longer be on the website.
M5	Create Child	2	As a Member, I want to register a child under my account, so that my child can access children's lessons safely.
M6	View Child	2	As a Member, I want to view my child's learning progress, so that so that I can support their participation
M7	Update Child	2	As a Member, I want to update my child's account settings, so that so that content stays age-appropriate.
M8	Delete Child	2	As a Member, I want to deactivate a child account, so that so that accounts that are no longer needed are no longer active.
M9	Create Meeting	2	As a Member, I want to schedule meetings with one of the pastors, so that I can meet with them one on one
M10	View Meeting	2	As a Member , I want to see my scheduled meetings, so that I can adjust if plans change.
M11	Update Meeting	2	As a Member, I want to reschedule or update meetings, so that I can adjust if plans change.
M12	Delete Meeting	2	As a Member, I want to cancel meetings, so that I can adjust if plans change.

ID	Story Title	Points	Description
M13	Create Prayer Request	2	As a/an Member , I want to create a prayer request (publicly or anonymously), so that the church can pray for me.
M14	View Prayer Request	2	As a Member, I want to view my current prayer requests, so that I can see what requests I have currently sent in and previously.
M15	Update Prayer Request	2	As a Member, I want to edit my prayer request , so that I can update information.
M16	Delete Prayer Request	2	As a Member , I want to delete my prayer request, so that the pastors/leaders never see it before it gets to them.
M17	Stream Sermons	2	As a Member, I want to stream audio and video sermons, so that I can engage with church messages remotely.
M18	Search Sermons	2	As a Member, I want to search sermons by speaker, topic, or series, so that I can easily find relevant messages.
M19	Comment on Sermon	1	As a Member, I want to comment on a sermon, so that I can share my thoughts.
M20	Update Comment	1	As a Member, I want to edit a comment, so that I can change what I commented.
M21	Delete Comment	1	As a Member, I want to delete a comment on a sermon, so that I can remove my comment if I want.
M22	Like a Sermon	1	As a Member, I want to like a sermon, so that I can share my thoughts.
M23	Remove Like	1	As a Member, I want to unlike a sermon, so that I can share my thoughts.
M24	Log In	1	As a Member, I want to log into my account, so that I can access specialized features.
M25	Log Out	1	As a Member, I want to log out of my account, so that I keep my information secure.
M26	Reset Password	2	As a Member, I want to reset my password, so that I can regain access if I forget it.
M27	Send Pastoral Appointment Reminders	1	As a Member, I want to receive reminders for pastoral appointments, so that I do not forget.
M28	Push Notifications	1	As a Member, I want to receive push notifications, so that I can be alerted for new sermons, prayer updates, scheduled meetings, or new children's lessons.
M29	Upload Profile Picture	1	As a Member, I want to upload a profile picture, so that people know who I am.
P1	Availability Input	2	As a Pastor, I want to enter my availability, so that I can be scheduled efficiently.
P2	Approve/Decline Meetings	2	As a Pastor, I want to approve or decline meeting requests, so that my schedule is manageable.
P3	View Schedule	2	As a Pastor, I want to view my scheduled meetings, so that I can meet with members on time.
P4	Cancel Meetings	2	As a Pastor, I want to cancel scheduled meetings, so that changes are reflected.
P5	View Prayer Requests	2	As a Pastor (and prayer team member), I want to view all incoming prayer requests, so that I can complete them.
P6	Mark Requests as Done	2	As a Pastor (and prayer team member), I want to mark prayer requests as complete, so that I can let the member know I have completed their request.
C1	Weekly Lessons	2	As a Child, I want to view weekly lessons and memory verses, so that I can learn about faith in a fun and engaging way.

ID	Story Title	Points	Description
C2	Complete Quizzes	3	As a Child, I want to complete quizzes after lessons, so that I can practice and remember what I learned.
C3	Learning Progress	3	As a Child, I want to see my learning progress, so that I feel motivated to continue participating.
T1	Monitor Child Progress	2	As a teacher, I want to view each child's progress so that I can support their learning effectively.
T2	Update Weekly Lessons	3	As a teacher, I want to create and update weekly lessons and memory verses so that content stays relevant and engaging.
T3	Manage Quizzes	3	As a teacher, I want to create and edit quizzes so that I can assess children's understanding of each lesson.
T4	Review Quiz Results	2	As a teacher, I want to review quiz results so that I can identify areas where children are struggling.
A1	Upload Audio Sermon	3	As an Administrator, I want to upload audio sermons, so that I can engage with church messages remotely.
A2	Upload Video Sermon	3	As an Administrator, I want to upload video sermons, so that I can engage with church messages remotely.
A3	Edit Sermon	3	As an Administrator, I want to edit uploaded sermons, so that I can keep the sermons up to date.
A4	Delete Sermon	2	As an Administrator, I want to delete uploaded sermons, so that the content stays current.
A5	Deactivate Accounts	1	As an Administrator, I want to deactivate unused accounts, so that the current member accounts stay updated.
A6	Role Changes	1	As an Administrator, I want to change the roles of members depending on what department they are part of.
A7	Send Announcement	1	As an Administrator, I want to send announcements or notifications, so that members stay informed.
G1	View Page	1	As a Guest, I want to view the landing page, so that I understand the church's mission.
G2	Stream Sermon	1	As a Guest, I want to stream audio and video sermons, so that I can engage with church messages remotely.
G3	View Info	1	As a Guest, I want to view information about the church, so that I can learn about the church.
G4	Social Media	1	As a Guest, I want to view the church's different social media accounts, so that I can stay connected with them on different platforms.
G5	Contact Church	2	As a Guest, I want to contact the church, so that I can ask questions easily.

Table 2: Functional requirements as User Stories

4 System Design

4.1 Architecture

We will use the MVC structure to define how different components interact in our system. We like this model for our project because it is good at defining roles when users interact with data. The Model is what manages the data. The controller is the messenger between the model and the view and it handles the incoming requests using logic. The view is the interface that interacts with the user. The main modules would be User Management, Sermons, Prayer Requests, Scheduling, Children's Ministry, and Notifications.

4.2 Diagrams

The View points to the Model because the View needs to read data from the Model in order to display up-to-date information to the user. The Controller points to the Model because it is responsible for updating and managing that data and logic in response to user actions. The Controller points to the View because it controls which views are updated after processing user input, ensuring the user sees the current state of the system.

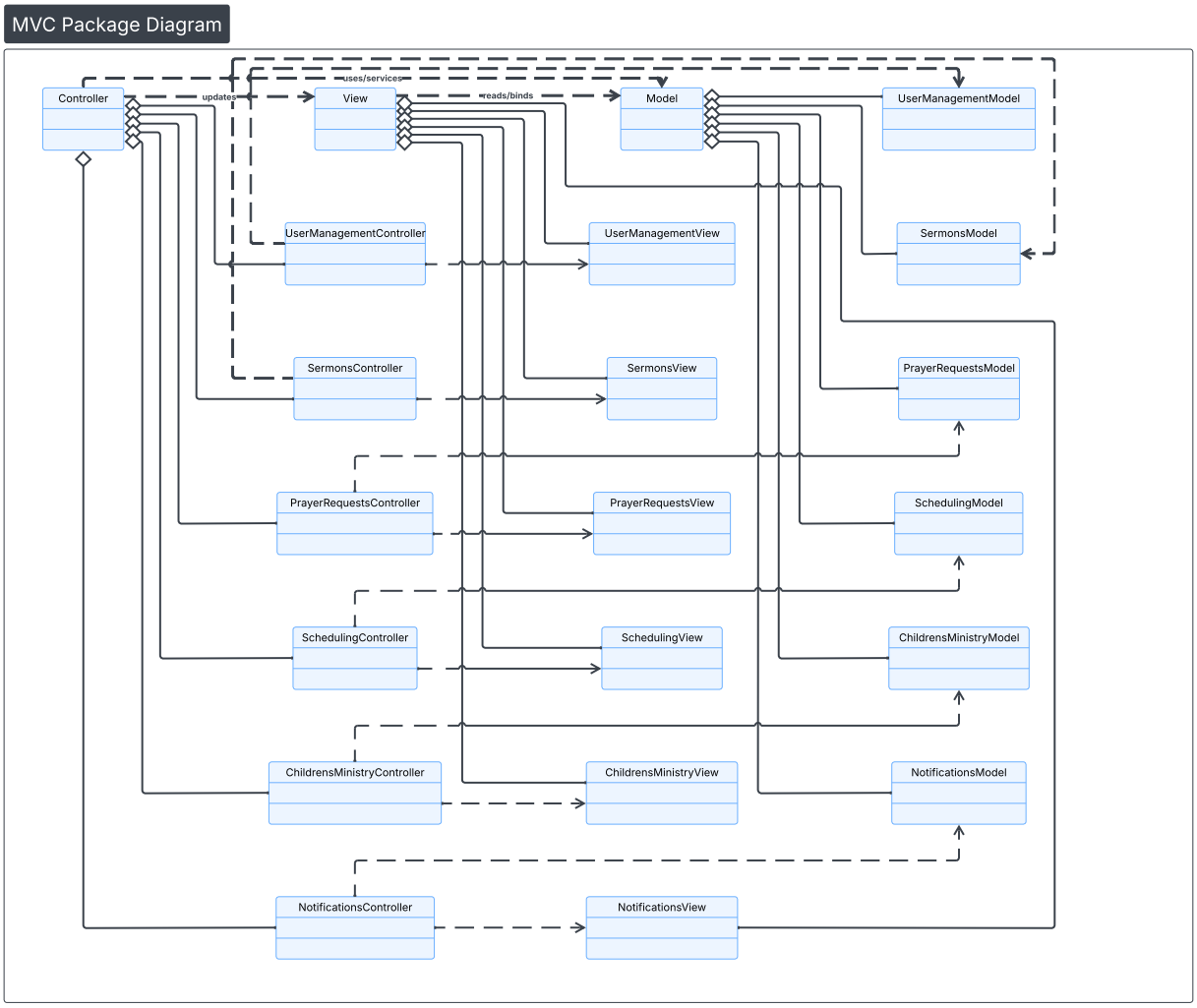

Figure 1: MVC Package Diagram for the Church Website Architecture

4.3 Technology

Table 3: Technology Stack Overview

Technology	**Purpose and Justification**
JavaScript	Main programming language. Used to implement controllers and Data Access Objects (DAOs) in a consistent and efficient manner. Supports clean code organization and aligns with the MVC architecture.
HTML	Frontend markup language. Chosen due to prior experience, enabling efficient development. Works seamlessly with CSS to maintain consistent page structure.
CSS	Used for styling and layout customization. Ensures a consistent look and feel across all pages.
React	Frontend library used to improve code re-usability, modularity, and web-page performance through component-based architecture.
Node.js	Backend runtime environment that executes JavaScript server-side.
Express.js	Backend framework used with Node.js to handle routing and manage web requests efficiently.
MongoDB	NoSQL database used to store application data in a flexible, document-based structure.
Mongoose	Object Data Modeling (ODM) library used to define schemas and enforce structure while maintaining MongoDB flexibility.
JSON	Data formatting standard used for structuring and transferring data between frontend and backend.
Playwright	Primary testing framework used to test authentication, sermon streaming, scheduling, and role-based access functionality.

4.4 Coding Standards

We will be following coding standards to ensure we maximize productivity between the 2 of us. We will follow our iteration planning that we decided on in the beginning of our work. We will follow the standard that we will stick to our iteration planning to our best ability, and if a change is needed to be made, we will consult our partner and correctly reflect the change in our documentation. We will release and push our code frequently, meaning that anytime a significant amount of progress in a user story is made or a user story is completed we will commit and push. We will require that every feature be accompanied by unit tests, and no code will be released unless all unit tests pass with at least 75% coverage. This ensures correctness, reliability, and prevents regressions throughout development. We will use camelCase for all javascript code and database collections will use lowercase singular names. To reduce merge conflicts, we will clearly divide tasks and communicate frequently to avoid working on the same files at the same time.

4.5 Data

We will be using a NoSQL database, as it is best suited for web applications and projects that evolve quickly due to its flexible schema structure. We will utilize MongoDB to store our data and take advantage of its schema flexibility as the project grows and requirements change. Mongoose will be used as an Object Data Modeling (ODM) library to allow our backend code to communicate efficiently with the database while still enforcing structure where necessary.

Collection	Key Attributes / Description
Members	ID, Email, First Name, Last Name, Role, Status
Leaders	Extends Member. Includes all Member fields plus: Managed Members, Scheduling Availability
Pastors	Extends Member. Includes all Member fields plus: Managed Members, Scheduling Availability
Children	Child-specific information (linked to Member accounts as needed).
Sermons	ID, Title, Speaker, Topic, Type, URL
Prayer Requests	ID, CreatedBy (nullable for anonymous submission), Text, Status
Appointments	ID, MemberID, PastorID, Time, Location, Status
Notifications	ID, UserID, Type, Contact Method, Message, TimeSent

Table 4: Database Collections and Key Attributes

4.6 UI Mocks

Figure 2: System screenshots showing key features of the application. The landing page, sign in page, and sermon media hub.

5 Iterations

5.1 Iteration Planning

Table 5 shows the iteration planning.

Iteration	Dates	Stories	Points
1	01/27 – 02/10	G1 View Page, G2 Stream Sermon, G3 View Info, G4 Social Media, G5 Contact Church, M1 Create Account, M2 View Account, M3 Update Account, M4 Delete Account (only 1 point completed), M24 Log In, M25 Log Out, NFR1 Access Control, NFR4 Consistent Layout	15
2	02/11 – 02/24	M4 Delete Account (remaining point), A1 Upload Audio Sermon, A2 Upload Video Sermon, A3 Edit Sermon, A4 Delete Sermon, M17 Stream Sermons, M18 Search Sermons, M19 Comment on Sermon, M21 Delete Comment, M22 Like Sermon, M23 Remove Like, M29 Upload Profile Picture, NFR5 Color Scheme	21
3	02/15 – 03/17	M13 Create Prayer Request, M14 View Prayer Request, M15 Update Prayer Request, M16 Delete Prayer Request	8
4	03/18 – 03/31	M9 Create Meeting, M10 View Meeting, M11 Update Meeting, M12 Delete Meeting, P1 Availability Input, P2 Approve/Decline Meetings, P3 View Schedule, P4 Cancel Meetings, P5 View Prayer Requests, P6 Mark Requests as Done, M27 Send Pastoral Appointment Reminders, M26 Reset Password, A6 Role Changes, A5 Deactivate Accounts, M20 Update Comment, NFR2 Session Security, NFR3 Data Persistence	28
5	04/01 – 04/14	M5 Create Child, M6 View Child, M7 Update Child, M8 Delete Child, T1 Monitor Child Progress, T2 Update Weekly Lessons, T3 Manage Quizzes, T4 Review Quiz Results, C1 Weekly Lessons, C2 Complete Quizzes, C3 Learning Progress, A7 Send Announcement, M28 Push Notifications, NFR6 Mobile Friendly, NFR7 Accessibility	26
		Total:	98

Table 5: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

For Iteration 1, our team planned a total of 16 story points. The objective of this iteration was ensuring the implementation of access to the core content, authentication, and user accounts, which enable the user to log in and out of the system completely.

In the case of guests, we included features like View Page, Stream Sermon, View Info, Social Media, Contact Church to ensure maximum user engagement even for users who have not registered on the site. These features clearly define the main purpose of the website to users.

At the same time, we concentrated on the essential members' functionality which includes implementing the "Create Account," "Log In," "Log Out," and full lifecycle management ("view," "update," "delete") functionalities of a member's account, which are precursors to the advanced members' functionalities we intend to design in the future.

5.2.2 Work Done

Zoe completed stories M1-M3 for a total of 6 story points. M4 can only be done by admin at the time so that story is only partially completed at the moment, for one more story point. Zoe completed a total of 7 story points.

Crystal completed G1-G5 for a total of 6 points and M24-M25 for 2 points. We both had contributions to access control and Crystal assured our landing page has consistent layout. Crystal completed a total of 8 story points. Together we completed 15 of the 16 planned story points.

5.2.3 Testing Coverage

At the moment, our coverage is not good enough. Our coding standard targets 60% coverage before release. In Iteration 1, we focused on establishing the project structure and verifying core processes like auth middleware and the user model. Overall coverage is currently below the 60% target because several modules were framed but not fully implemented/tested yet. In Iteration 2, we will prioritize adding integration tests for all auth and user endpoints and expand frontend component tests so that overall coverage increases.

```
PS C:\Users\z4rai\OneDrive - Loyola University Maryland\CS 496\glory-harbor-works\server> npm run test:coverage

> server@1.0.0 test:coverage
> node --experimental-vm-modules node_modules/jest/bin/jest.js --coverage

(node:24468) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/middleware/auth.test.js
  Auth Middleware
    √ should return 401 if no authorization header provided (8 ms)
    √ should be a function

(node:11260) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/models/User.test.js
  User Model
    User Schema Validation
      √ should create user with valid data (13 ms)
      √ should set default role to member (1 ms)
      √ should set default status to active
      √ should convert email to lowercase (1 ms)
      √ should have comparePassword method (1 ms)
      √ should require email, firstName, lastName, and password (4 ms)


(node:9776) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/controllers/authController.test.js
  Auth Controller
    register
      √ should be a function (6 ms)
    login
      √ should be a function (1 ms)
    logout
      √ should return logout success message
      √ should be a function (1 ms)

--------|---------|----------|---------|---------|-------------------
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
--------|---------|----------|---------|---------|-------------------
...iles |    8.86 |     1.36 |   11.76 |    8.91 |
 ...ers |    4.22 |        0 |    7.14 |    4.22 |
  ...js |   14.28 |        0 |      20 |   14.28 | ...66-101,107-114
  ...js |       0 |        0 |       0 |       0 | 7-354
 ...are |   55.55 |       25 |     100 |   55.55 |
  ...js |   55.55 |       25 |     100 |   55.55 | 11-15
 models |   42.85 |        0 |       0 |      50 |
  ...js |   42.85 |        0 |       0 |      50 | 29-30,35
--------|---------|----------|---------|---------|-------------------
Test Suites: 3 passed, 3 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        2.139 s
Ran all test suites.
```

Figure 3: Iteration 1 test coverage report.

5.2.4 Retroespective & Reflection

Zoe - When setting up the database in MongoDB I didn't know I had to set access to everywhere, so at first Crystal couldn't access it. It was a minor setback, but something that could have easily been avoided if I had read the directions or watched a video on how to set it up. Currently I have to run the server and client side with different commands and in different terminals so that is something we need to improve in the next iteration.

Crystal - As I work on the website, one of my main focuses is the aesthetic. I want the site to be visually appealing or at least look properly organized. I spent most of my time on the UI. I also worked on the

registration page but ran into trouble when I tried to test it. Once I got access from Zoe everything worked properly. I am enjoying implementing different UI features but I need to make sure I spend time on the actual user stories and not just the looks.

5.3 Iteration/Sprint 2

5.3.1 Planning

We plan to complete L1 Upload Audio Sermon, L2 Upload Video Sermon, M17 Stream Sermons, M18 Search Sermons, M22 Like Sermon, M23 Remove Like, and NFR5 Color Scheme this iteration for a total of 12 story points. This iteration is mainly focused on complete the sermon feature of our main modules in our MVC structure. We may implement more stories if time permits as this is our lowest point value iteration that we have planned, and our last iteration has the most points so we should do more now when we can.

5.3.2 Work Done

Zoe finished M4 from the last iteration for an additional point. Zoe also completed M17 to stream sermons, M18 to search sermons, M22 Like Sermon, and M23 Remove Like for a total of 7 story points. M17 was to create a sermon hub and add navigation to it. Now, the sermons act as more of a chat box than just youtube videos embedded on the main page. Zoe also had extra time and completed L3 and L4 to edit and delete sermons for an additional 5 points making Zoe's total story points 12 for this iteration.
Crystal completed L1 Upload Audio Sermon, L2 Upload Video Sermon, and finished up G3 View Info from the last iteration as well as some fixes/design changes. Crystal also extra time and completed M19 Comment on Sermon, M21 Delete Comment, and M29 Upload Profile Picture that we had planned to be completed in later iterations. Her total story points for this iteration is 9. Authorized users can upload and edit sermons and logged in users are able to listen to the sermons and comment on them. Users can upload a pfp that is visible when they comment. Together, they completed 21 story points, so 9 additional points past the planned 12 points. This is good for the team as we want to have fewer points planned down the line.

5.3.3 Testing Coverage

Our coverage is much better than the last iteration. Although, we can still improve much more especially in branch coverage. We need to add more targeted method branch tests to increase User.js coverage.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	32.55	20.39	39.21	32.87	
controllers	31.27	21.42	41.46	31.55	
authController.js	14.28	0	20	14.28	7,14-62,68-105,111-118
contactController.js	79.31	86.66	100	79.31	34-35,58-59,91-92
prayerController.js	0	0	0	0	4-87
sermonController.js	19.7	10.25	29.41	20.1	...9-240,269-337,355-377,386-435,440-483,499-535,555-578
userController.js	50.69	41.33	66.66	50.69	...2,257,263,275-276,289-330,355-367,393-399,432,444-445
middleware	38.29	9.09	37.5	38.29	
auth.js	30	8.33	50	30	12-16,21-31,41-57
upload.js	52.94	10	25	52.94	11,17-22,27-31
models	53.33	0	0	57.14	
Appointment.js	0	100	100	0	3
Contact.js	100	100	100	100	
Notifications.js	0	100	100	0	3
Prayer.js	0	100	100	0	3
Sermon.js	100	100	100	100	
Prayer.js	0	100	100	0	3
Sermon.js	100	100	100	100	
User.js	42.85	0	0	50	30-31,36

Test Suites: 12 passed, 12 total
Tests: 67 passed, 67 total
Snapshots: 0 total
Time: 3.419 s
Ran all test suites.

Figure 4: Iteration 2 test coverage report.

5.3.4 Retrospective & Reflection

Zoe - The biggest challenge for us for this iteration was testing. We had to catch up on implementing all testing from the last iteration, and it piled up and ended up being the most time consuming task this iteration. We realized that we need to be writing our tests as we implement different modules of our webpage so that we can test after we build a certain feature and not wait for the end to do all testing. Also we could've added more points to this iteration rather than having more points in our last iteration.
Crystal - This iteration didn't include as many points as the previous one. Although we completed everything that was planned, we should have worked on upcoming iterations to lessen our future workload. Going forward, I will begin stories earlier so I have enough time to polish the UI, make sure everything works properly and, when possible, work on future iterations.

5.4 Iteration/Sprint 3

5.4.1 Planning

We plan to complete M13 Create Prayer Request, M14 View Prayer Request, M15 Update Prayer Request, M16 Delete Prayer Request, P5 View Prayer Requests, P6 Mark Requests as Done, M26 Reset Password, M27 Send Pastoral Appointment Reminders, G6 Register with Invite Code, and NFR2 Session Security for a total of 17 story points. In this iteration, we are focusing on the prayer request system and fully implementing the MVC structure for this module.

5.4.2 Work Done

Zoe Completed M13-M16 for 8 story points. All four stories were completed end-to-end in this iteration, including backend controller logic, route wiring, and automated tests to validate core success and error paths.

Crystal did not fully complete any of the user stories. Instead, she focused on making changes that the client asked for.

5.4.3 Testing Coverage

Our current testing coverage is 67.98% for statements, 63.91% for branches, 71.92% for functions, and 68.04% for lines, with 16 test suites and 126 tests all passing. This is a solid improvement from the last iteration which shows that we have made significant progress, especially on some critical files such as the prayer, user, and contact controllers, but we still need to focus on weaker areas such as authentication, upload middleware, as well as some models. In the next iteration, we intend to continue writing tests as we go so that we do not end up with a large pile-up at the end.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	67.98	63.91	71.92	68.04	
controllers	73.12	73.43	84.44	73.11	
authController.js	14.28	0	20	14.28	7,14-62,68-105,111-118
contactController.js	79.31	86.66	100	79.31	34-35,58-59,91-92
prayerController.js	82.97	100	100	82.97	13-14,40-41,78-79,110-111
sermonController.js	74.36	70.65	85	74.35	...6,521,536-537,588-589,631-632
userController.js	82.42	87.35	100	82.42	...7-378,414-415,446-447,492-493
middleware	31.94	3.92	30	31.94	
auth.js	30	8.33	50	30	12-16,21-31,41-57
upload.js	33.33	2.56	16.66	33.33	11,17-22,27-31,48-59,63-86
models	62.5	0	0	66.66	
Appointment.js	0	100	100	0	3
Contact.js	100	100	100	100	
Notifications.js	0	100	100	0	3
Prayer.js	100	100	100	100	
Sermon.js	100	100	100	100	
User.js	42.85	0	0	50	30-31,36

```
Test Suites: 16 passed, 16 total
Tests:       126 passed, 126 total
Snapshots:   0 total
Time:        5.127 s
Ran all test suites.
```

Figure 5: Iteration 3 test coverage report.

5.4.4 Retrospective & Reflection

Zoe - In this iteration, I think I became much more comfortable with the MVC pattern and how all three layers interact with one another. Doing this prayer request feature really helped me understand things like storage in the database, validation, error checking, etc., as well as ensuring that only the user can edit or delete their own prayer requests. One of my biggest struggles again was testing, as we still had to put a lot of time into this, but this time we were able to get to a much better level of testing. I think this has really taught me the importance of testing each feature as we go instead of waiting until the end, as this makes things much less overwhelming.
Crystal - This iteration I was not able to contribute much. After communication with the client I had to go back and try to make changes to the registration and roles. I ran into many roadblocks along the way and my code didn't work as intended. This was eye opening and made me realize how important it is to have constant communication with the client. Going forward I will make sure to check in with the client as I make changes.

5.5 Iteration/Sprint 4

5.5.1 Planning

We plan to create the scheduling/meeting module in this iteration which includes M9 Create Meeting, M10 View Meeting, M11 Update Meeting, and M12 Delete Meeting. We will also complete pastor stories, P1 Availability Input, P2 Approve/Decline Meetings, P3 View Schedule, P4 Cancel Meetings. In addition, we plan to complete M27 Send Pastoral Appointment Reminders, M20 Update Comment, and NFR3 Data Persistence. We also did not complete P5 View Prayer Requests, P6 Mark Requests as Done, M26 Reset Password, G6 Register with Invite Code, and NFR2 Session Security from the previous iteration. This would total 26 points.

5.5.2 Work Done

Zoe - I completed M9 to M12 for the full CRUD of scheduling meetings with pastors. This will complete the meetings module from the member side. I also completed the M20 update comment on a sermon. I also completed P1-P4 to complete the full CRUD for the scheduling system from the pastor side. I also completed M26 to reset the password. I completed a total of 19 points. I also implemented NFR3 by adding stronger database reliability and recovery support, not just basic MongoDB storage. The backend now checks database connectivity before booting up and includes backup and restore scripts so saved data

and uploaded files can be recovered after failures or restarts.
Crystal - I incorporated Calendly's API for meeting scheduling. Through Calendly, members can schedule meetings and receive automatic notifications. The pastor can also set availability and cancel meetings, though most of this is handled through the app. I also completed NFR2, A6, A5, P5, and P6, and redesigned the registration process so it no longer requires an invite code. Roles are now assigned by an admin rather than during registration. After speaking with the client, I implemented the changes that best suited their needs. Additionally, I made several updates outside of our original user stories based on client requests. These include improvements to the sermon search function, adding a "top of page" button on the About Us page, and creating hyperlinks for all references to the church's email, address, and phone number, among other enhancements.

5.5.3 Testing Coverage

We realized that our original coding standard of 60% is a bit low and easily achievable, so we have increased that to 75%. The main testing that we implemented this iteration was for the appointment model and controller. Some files still have lower coverage, especially middleware and smaller supporting artifacts, because they are harder to isolate and test directly, but in the next iteration we plan to add more targeted tests for those areas to make our coverage more balanced.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	84.06	78.86	88.57	83.97	
controllers	89.01	86.79	98.27	88.95	
appointmentController.js	96	90	100	96	20,24,34,42,46,61
authController.js	90.47	100	100	90.47	62-63,107,120
contactController.js	79.31	86.66	100	79.31	34-35,58-59,91-92
prayerController.js	82.97	100	100	82.97	13-14,40-41,78-79,110-111
sermonController.js	90.6	81.06	95.23	90.45	...361,377,415,418,517,521,526
userController.js	84.02	88.78	100	84.02	...451,487-488,519-520,565-566
middleware	31.94	3.92	30	31.94	
auth.js	30	8.33	50	30	12-16,21-31,41-57
upload.js	33.33	2.56	16.66	33.33	11,17-22,27-31,48-59,63-86
models	93.75	100	100	93.33	
Appointment.js	100	100	100	100	
Contact.js	100	100	100	100	
Notifications.js	0	100	100	0	3
Prayer.js	100	100	100	100	
Sermon.js	100	100	100	100	
User.js	100	100	100	100	

Test Suites: 19 passed, 19 total
Tests: 234 passed, 234 total
Snapshots: 0 total
Time: 10.106 s
Ran all test suites.

Figure 6: Iteration 4 test coverage report.

5.5.4 Retroespective & Reflection

Zoe - the biggest challenge this iteration was actually trying to achieve data persistence, one of our non-functional requirements. The main pitfall was that the system could save data but didn't have anyway to recover data or handle startup failure. I added database health monitoring, fast startup help, and backing up data so we could achieve persistence. It was surprising how much work this was and how particular I had to be. Adding the code for this was different than what we have been doing with MVC where things by now have gotten pretty similar module to module. This NFR challenged me to think more systemically in addition to feature development. In the next iteration, we can improve the process by planning more accordingly for the NFR's we plan to complete and allocating more time to these systematic details rather than just core features. This experience taught me how different the system level is compared to our usual MVC structure, where the pattern is much more repetitive.
Crystal - This iteration, I had an idea of what I wanted to do, but actually implementing it was a challenge. One of the biggest things I struggled with was integrating the Calendly API. At first, I assumed it would be a simple plug-in type of process, but it ended up being more complex than I expected. I had trouble understanding how the authentication worked and how to properly connect it to the site. There were also

times where the data I got back didn't match what I thought it would look like. I had to do a lot of testing as I went along and it's still not completely what I had envisioned. This experience showed me that I need to give myself more time when working with external tools and not assume things will be straightforward. On the other hand, I enjoyed creating the front end and designing the look of the site. That is the most enjoyable part of the process for me.

5.6 Iteration/Sprint 5

5.6.1 Planning

We plan to complete M5 Create Child, M6 View Child, M7 Update Child, M8 Delete Child, T1 Monitor Child Progress, T2 Update Weekly Lessons, T3 Manage Quizzes, T4 Review, Quiz Results, C1 Weekly Lessons, C2 Complete Quizzes, C3 Learning Progress, A7 Send Announcement, and M28 Push Notifications for a total of 21 story points. In this iteration, we are focusing on the full Child CRUD and child account functionalities. We are also finishing up any additional NFR's or leftover stories as this is our final opportunity to polish our project.

5.6.2 Work Done

Zoe - I completed C1-C3 to implement the child functionalities for 8 points. I also completed M28 and A7 for an additional 2 points. I complete 10 total points this iteration.
Crystal - This iteration I completed M5 Create Child, M6 View Child, M7 Update Child, M8 Delete Child, T1 Monitor Child Progress, T2 Update Weekly Lessons, T3 Manage Quizzes, and T4 Review Quiz Results for a total of 16 points.

5.6.3 Testing Coverage

Testing is very important for our final iteration because it gives us confidence that the main user stories work correctly and that future changes will not easily break existing features. Our backend test coverage is strong across the main controllers, models, and middleware, with 100% coverage for meetingController.js, lessonController.js, and auth.js, and improved branch coverage for calendlyController.js from 43.23% to 80.78%. We accepted lower coverage for UI styling and external API behavior that depend on browser rendering or third-party services like Calendly, focusing our automated tests on code paths we can reliably control with mocks. Overall, we are happy with our final testing coverage percentages because the most important business logic, authorization checks, validation rules, and error handling paths are tested.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	89.15	85.82	93.75	89.24	
controllers	88.55	85.51	93.29	88.64	
appointmentController.js	96	90	100	96	20,24,34,42,46,61
authController.js	72.41	76.66	83.33	72.41	67-68,114,127,132-151
calendlyController.js	91.7	80.78	83.87	92.34	101-117,122-124,317-318
childController.js	67.18	56.71	91.66	68.03	...220,224,234-235,245,249,258-259
contactController.js	79.31	86.66	100	79.31	34-35,58-59,91-92
lessonController.js	100	99.01	100	100	83
meetingController.js	100	98.52	100	100	62,84,299
prayerController.js	54.66	71.42	57.14	54.66	...3,80-81,112-113,122-143,152-187
sermonController.js	90.87	81.01	95.65	90.67	...369,373,389,427,430,529,533,538
userController.js	84.48	87.4	100	84.48	...535-536,597-598,619,626,648-649
middleware	98.61	92.15	100	98.61	
auth.js	100	100	100	100	
upload.js	97.61	89.74	100	97.61	56
models	100	100	100	100	
Appointment.js	100	100	100	100	
CalendlyMeetingAction.js	100	100	100	100	
Child.js	100	100	100	100	
ChildLessonProgress.js	100	100	100	100	
Contact.js	100	100	100	100	
Lesson.js	100	100	100	100	
Meeting.js	100	100	100	100	
Notifications.js	100	100	100	100	
Prayer.js	100	100	100	100	
Sermon.js	100	100	100	100	
User.js	100	100	100	100	

```
Test Suites: 30 passed, 30 total
Tests:       375 passed, 375 total
Snapshots:   0 total
Time:        12.377 s
Ran all test suites.
```

Figure 7: Iteration 5 test coverage report.

5.6.4 Retrospective & Reflection

Zoe - The one major obstacle this iteration faced in adapting to change was due to alterations in our role system that meant changes to login and dashboard functionality had to be made and that forced us to evaluate whether certain user stories were still necessary. While better initial planning regarding our system would have been helpful, several of our problems were solved once we began putting the puzzle pieces of other features into place. For example, this iteration I worked on the notification system. Getting notifications to interact smoothly with our current application meant getting them to integrate with announcements, profiles, sermons, lessons, prayers, and meetings, all while figuring out how to store our data, how to deliver notifications by user role, and how not to break everything else in the process. Throughout this process, I realized the importance of testing for the purpose of catching minor logic errors when integrating features that require multiple user roles, records in the database, and varying levels of permissions.
Crystal - For me, this iteration went quite smoothly. Planning the data models early made the co-parent invite flow and quiz progress tracking a lot easier to implement. Since I thought through the edge cases before writing any code, most things just worked the first time. The hardest part was authorization and making sure the right parent could do the right thing and that teachers couldn't accidentally leak correct answers before a quiz was finished. That required me to constantly log in as different users and check to see that each role could only do what they were supposed to. Overall this was my busiest iteration, but finishing all 8 stories on time showed me how important it is to start with a solid plan.

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]